PSAI_145

Tux0

- **Team Name:** Libeccio Labs
- **Payment Address:** 12poSUQPtcF1HUPQGY3zZu2P8emuW9YnsPduA4XG3oCEfJVp (USDT on Asset Hub)
- **Level:** 2 🐈‍⬛
- **Status:** Terminated

Project Overview :page_facing_up:

A Tuxedo piece that leverages zero-knowledge proofs to manage a private token.

Overview

Zero-knowledge proofs are revolutionizing the way we interact with the blockchain. They can be used to create new types of applications that are more private and secure.
We are particularly interested in finding a proving system which does not require a trusted setup, and is good enough to be used in a Substrate runtime.
Truth is zero knowledge primitives on parachains as host functions are still far, since it is a long and tedious path to decide, implement, test, and bring them on the relay chain, which involves Parity and the Polkadot Fellowship.

Every developer in the Substrate ecosystem can benefit from this research. If we are successful in figuring out a fast and efficient zk protocol for Substrate runtimes, it will open up a whole new range of possibilities for runtime developers, particularly for parachains.

A DAP (decentralized anonymous payment) is the simplest, yet useful, implementation of zero-knowledge proofs in blockchain we could think of, because it only needs to prove that the sender has the private key corresponding to the output UTXO and that the transaction is valid.

We naturally chose to build on Tuxedo because it is designed to make it easy to develop UTXO-based blockchains, which DAPs typically are. For example, Manta Network implemented UTXOs inside a FRAME pallet for the same purpose, which is not ideal.
We believe that an UTXO-based alternative to FRAME was needed, to provide developers with more choices and flexibility, leading to innovation and experimentation, which can benefit the entire ecosystem.
As a consequence of this choice, this project will prove that Tuxedo is ready for people to write their own runtimes, and hopefully soon their parachains as well.
However, the objective of this project is not to develop a production-ready module, but to conduct a research and showcase our results to the Substrate community as a whole.

Project Details

Research phase

The first part of the project will involve a research on zero knowledge protocols, with the objective of finding out a protocol that can potentially be used in a parachain runtime. We're particularly interested in non-interactive protocols that don't require a trusted setup, use succinct proofs, are fast to verify.
Some potential candidates are: - Halo2, used by Zcash in its orchard protocol; - Plonky2, developed by Polygon Zero; - Kimchi, by O(1) Labs, used by the Mina Protocol.

The zero knowledge protocol used in the second phase will be chosen based on this research's results. The factors that will be considered are, by importance: - verification performance and proof size; - proof construction performance; - security, if the library is production-ready; - ease of use, for example high level languages to write circuits.

Proof construction performance and proof size will be measured by running the construction function on a set of pre-generated inputs, and measuring the average time it takes to construct each proof. The prover will be tested in both native and WASM, as the wallet is currently only available as a native binary, but we think it would make sense to have a library that can be used in-browser as well.

Verification performance will be measured by running the verification function on the previously generated proofs, and measuring the average time it takes to verify each proof. The verifier will need to be compiled to WASM, since it will be used in a Substrate runtime.
The execution times of some real-world FRAME extrinsics will be measured and included as well, as we hope to find a protocol with comparable execution times.

We are aware that the results might not be as good as expected. If we fail to identify a protocol that can be used in a Substrate runtime, then we would still proceed with developing the final product, using custom host functions.
However, since some of these protocols are already used in other blockchains, we're confident this phase will be a success.

Benchmarking program architecture

There will be a sovereign trait to be implemented for each protocol one wants to benchmark.
One can also extend the trait to add more test cases to the benchmarking program. Something along the lines of:

```rust
pub type Proof = Vec<u8>;

pub trait Protocol {
    /// Build a proof for the sum of two numbers
    fn sum_build_proof(a: i64, b: i64, sum: i64) -> Proof;
    /// Verify a proof for the sum of two numbers
```

```
        fn sum_verify_proof(proof: Proof) -> bool;

        /// Build a proof for a Sudoku
        fn sudoku_build_proof(sudoku: Sudoku) -> Proof;
        /// Verify a proof for a Sudoku
        fn sudoku_verify_proof(proof: Proof) -> bool;

        // ... other tests ...

        pub fn run_benchmarks() -> Vec<BenchmarkingResult> {
            let sum_proof = Self::sum_build_proof(2, 1, 3);
            // ... run all the benchmarks ...
        }
    }
```

We will implement a `run_benchmarks` method for every type that implements the Protocol trait, and we will provide a blanket implementation of Protocol for tuple of types that implements Protocol.

```
    type Protocols = (Halo2, Plonky2, Kimchi);

    fn main() {
        let benchmarking_results = Protocols.run_benchmarks();
        // ... export the results ...
    }
```

With this system, integrating new protocols will be as easy as implementing the Protocol trait for them, and adding a new item to the Protocols type.

Development phase

The final product will be a Tuxedo piece, written in Rust, which can be used in any Tuxedo runtime for private token transactions. The code will be stored in a public GitHub repository, along with an example on how to use it in a Tuxedo runtime. The documentation will be generated from Rustdocs and hosted on GitHub pages.
We will also develop an extension for Tuxedo's wallet, to be able to retrieve the balance for a certain address, build zero knowledge proofs and send transactions.
We will write unit tests for all of the Tuxedo primitives we develop for Tux0 to ensure that they are functionally correct, along with integration tests to show that the Tux0 wallet can successfully create and verify transactions against a template runtime.

The basic primitives behind the protocol we designed for Tux0 are:
- **Coin**, a simple public currency, managed by the Money piece. Each UTXO has a certain monetary value, and optionally an owner.
Wallets need to check all the transactions, to see if any produced UTXO is owned by a certain address they manage. Using the same process, any external entity can easily figure out the balance and transaction history for any public key. - **DAP Coin**, similar to Coin, but it is stored on-chain as a H256, encrypted using the owner's public key. Each UTXO has a private monetary value, an owner, and other parameters needed for proving its ownership and uniqueness.
Wallets will need to check all the transactions to see if any produced DAP Coin can be decrypted and decoded. However, external entities will not be able to understand the senders, recipients and values of any transaction, ensuring privacy.
- **Transfer function**, the only transaction the Tux0 piece will allow, which can consume and produce Coin or DAP Coin indiscriminately. As a consequence, users can freely convert Coin to DAP Coin and vice versa.
Each DAP Coin in input requires a zero knowledge proof to demonstrate the sender has the right to consume it, without revealing its identity.
Another zero knowledge proof is needed to prove that the operation is valid, because the sum of the outputs is less or equal to the sum of the inputs, without revealing any of these numbers.
Due to constraints imposed by the chosen proving system, this function will have a limited amount of inputs and outputs.

We can't guarantee for the project's security, since its purpose is to demonstrate the use of the chosen zero knowledge protocol, not to write a production-ready module. For that, the protocol should be thoroughly reviewed by experts, and the libraries and piece should be subject to security audits.

Ecosystem Fit

Our research will be useful to the Substrate community because it will help to identify zero-knowledge protocols that are suitable for use in Substrate runtimes.
This could enable Parachain developers to use zero-knowledge proofs for any use-case they want, without needing to wait for primitives to be added to the Relay Chain.

After a proper security audit, Tux0 would be a valuable tool for developers who want to build privacy-preserving dapps on the Polkadot/Substrate/Kusama ecosystem. It would also be useful to users who want to be able to send and receive tokens privately.
Moreover, Tux0 would demonstrate that Tuxedo is ready for developers to build their custom logic on top of it, and hopefully to be able to use it in production soon.

Even if fundamentally different, the protocol we designed for Tux0 is inspired by the Zerocash protocol, which uses zero knowledge proofs for private transactions.
An example of zero knowledge proofs used for private transactions can be found in Manta Network, where the protocol used requires a trusted setup. Moreover, their private coin is a simulation of UTXOs written inside a FRAME pallet, while Tuxedo allows native UTXOs in a Substrate runtime, and we are sure that our transactions costs will be fundamentally lower.

Team :busts_in_silhouette:

Team members

- Matteo Muraca
- Alberto Perrella

Advisors: - Joshy Orndorff

Contact

- **Contact Name:** Matteo Muraca
- **Contact Email:** muraca.matteo@icloud.com

Legal Structure

- **Registered Address:** N/A
- **Registered Legal Entity:** N/A

Team's experience

Matteo and Alberto were the first hires at NFT Chain, and worked there until its termination of business. Matteo successfully graduated at the second cohort of the Polkadot Blockchain Academy in Buenos Aires, after which he started contributing to the Polkadot SDK, and lately to Tuxedo as well. At NFT Chain, he was one of the core developers for a parachain to handle custom non-fungible assets with formal constraints.
Alberto holds a Master's Degree in Mathematics at ETH Zurich. At NFT Chain, along his core team duties, he was responsible for prototyping a protocol that leverages zero knowledge proofs to facilitate the verification of formal constraints on-chain.

Unfortunately, all the work done at NFT Chain is private, and we're legally obliged to not share it.
However, we presented some working prototypes at Sub0 2022, and the project has been accepted as part of the Substrate Builders Program.

Joshy Orndorff is one of the main contributors to Tuxedo. He's entered the Substrate ecosystem in 2019, and among all things he also taught the Polkadot Blockchain Academy in all three cohorts: Cambridge, Buenos Aires, and Berkeley.
He will not be directly involved in the development of this project, but he will be available for advice and guidance, mostly on the Tuxedo side.

Team GitHub Profiles

- https://github.com/muraca
- https://github.com/Gorzorg

The eventual code and documentation will be published in the Libeccio Labs GitHub organization: - https://github.com/LibeccioLabs

Team LinkedIn Profiles

- https://www.linkedin.com/in/matteo-muraca/
- https://www.linkedin.com/in/alberto-perrella-2522bb1a2/

Development Status :open_book:

We have currently identified some zero knowledge protocols which might become candidates for the research part.
We also started sketching out some code for a Tuxedo piece, but nothing worth mentioning.

Development Roadmap :nut_and_bolt:

Overview

- **Total Estimated Duration:** 20 weeks
- **Full-Time Equivalent (FTE):** 1.5 FTE
- **Total Costs:** 30,000 USD

Milestone 1 — Research phase, part 1

- **Estimated duration:** 8 weeks
- **FTE:** 1.5
- **Costs:** 12,000 USD

Number	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	We will provide both **inline documentation** of the code and a basic **tutorial** that explains how a user can run the proofs.
0c.	Testing and Testing Guide	The circuits will be tested with valid and invalid witness values. In the guide, we will describe how to run these tests.
0d.	Docker	We will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
1.	Demo proofs	We will implement 2 zero knowledge proofs (*Sudoku* and *Factorial*) for 2 protocols. These proofs will be later integrated in the benchmarking program defined in milestone 2.
		We will publish two **articles** on our GitHub blog, one for each protocol, documenting our development journey, highliting

2.	Articles on protocols	pros and cons, along with general comments about usability and suggestions for developers who want to use them.

Milestone 2 — Research phase, part 2

- **Estimated duration:** 6 weeks
- **FTE:** 1.5
- **Costs:** 9,000 USD

Number	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	We will provide both **inline documentation** of the code and a basic **tutorial** that explains how a user can add a new protocol, run the benchmarks, and verify the results.
0c.	Testing and Testing Guide	Core functions will be fully covered by comprehensive unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests.
0d.	Docker	We will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
1.	Demo proofs	We will implement 2 zero knowledge proofs (*Sudoku* and *Factorial*) for a third protocol.
2.	Article on protocol	We will publish an **article** on our GitHub blog documenting our development journey with the third protocol, highliting pros and cons, along with general comments about usability and suggestions for developers who want to use them.
3.	Benchmarking program	We will develop a Rust program that will automatically run and benchmark a predetermined set of zero knowledge proofs (at least a *Sudoku* and a *Factorial*) and export the results in a readable format, like JSON.
4.	Protocols integration	We will provide an integration of the 3 protocols and the previously developed Demo Proofs for the benchmarking program, which will be used to produce the results.
5.	Data Visualization tool	We will provide a single page webapp to easily visualize and compare the benchmark results, using C3.js or a similar library.
6.	Article on results	We will publish an **article** on our GitHub blog that explains our research process, the results, and why we decided to proceed with a certain protocol.

Milestone 3 — Development phase

- **Estimated Duration:** 6 weeks
- **FTE:** 1.5
- **Costs:** 9,000 USD

Number	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	We will provide both **inline documentation** of the code and a basic **tutorial** that explains how Tux0 can be included in a Tuxedo runtime, and how to interact with it, using Tuxedo's Template Wallet.
0c.	Testing and Testing Guide	Core functions will be fully covered by comprehensive unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests.
0d.	Docker	We will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
1.	Tux0	We will develop a Tuxedo piece that manages a DAP token using zero knowledge proofs. It will be written in Rust, and it will leverage the library identified in the first milestone.
2.	Tuxedo Wallet Extension	We will extend the Tuxedo Template Wallet to support Tux0 balances and to send Tux0 transactions. This will also be written in Rust.

Future Plans

We intend to continue to maintain Tux0 at least until a proper release of Tuxedo's parachain support - which might even come before the delivery of this grant.
We are interested in testing the zero-knowledge protocol we chose for Tux0 in a parachain environment.

Bader @CrackTheCode016 suggested we could deploy a Tux0 parathread on Rococo, which would be a huge step forward for Tuxedo as well.

Referral Program (optional) :moneybag:

- **Referrer:** Joshy Orndorff
- **Payment Address:** 0x5a335908df9D2C47304338E3b744579Ed7C6a64d (DAI)

Additional Information :heavy_plus_sign:

We have been part of the Substrate ecosystem since early 2022, so we had the chance to hear about the W3F Grants from various sources: Substrate Builders Program, Polkadot Blockchain Academy, Parity employees, Sub0 2022, and probably more.
However, it was Joshy the one who suggested us to ask for a grant for this idea, and he deserves a big thank you from both of us.